

Najczęściej stosowane diagramy UML

Michał Wolski / 2025-06-16 / <https://wolski.pro/>

W dzisiejszym świecie IT, gdzie projekty stają się coraz bardziej złożone, a zespoły często pracują zdalnie i są rozproszone geograficznie, potrzeba jednoznacznej komunikacji technicznej jest kluczowa. UML (Unified Modeling Language) od lat pozostaje jednym z najważniejszych narzędzi w arsenale inżyniera oprogramowania, oferując standardowy sposób wizualizacji i dokumentowania systemów informatycznych.

Kluczowe zalety stosowania UML w procesie wytwórczym oprogramowania

1. Standaryzacja komunikacji w zespole projektowym

UML działa jak uniwersalny język techniczny, który rozumieją zarówno analitycy biznesowi, architekci systemów, jak i programiści. Dzięki standardowym notacjom wszyscy członkowie zespołu mogą efektywnie komunikować swoje pomysły i zrozumienia systemu, niezależnie od swojego doświadczenia czy specjalizacji. To szczególnie ważne w międzynarodowych projektach, gdzie różnice kulturowe i językowe mogą utrudniać współpracę.

2. Jednoznaczność interpretacji wymagań i projektu systemu

Diagramy UML eliminują dwuznaczności, które często pojawiają się w dokumentacji tekstowej. Wizualna reprezentacja relacji między obiektami, sekwencji operacji czy przepływów danych pozwala na precyzyjne określenie tego, jak system powinien działać. To znacznie redukuje ryzyko błędnej interpretacji wymagań na etapie implementacji.

3. Możliwość prezentacji systemu na różnych poziomach abstrakcji

UML oferuje różne typy diagramów, które pozwalają na przedstawienie systemu z różnych perspektyw – od wysokopoziomowej architektury po szczegóły implementacyjne. To umożliwia dostosowanie komunikacji do potrzeb konkretnych odbiorców: kierownictwo może skupić się na diagramach przypadków użycia, podczas gdy zespół deweloperski będzie analizować diagramy klas i sekwencji.

4. Wsparcie dla całego cyklu życia oprogramowania

UML nie jest narzędziem używanym tylko na początku projektu. Diagramy ewoluują wraz z systemem, wspierając proces od analizy wymagań, przez projektowanie i implementację, aż po utrzymanie i rozwój. W Enterprise Architect możemy śledzić zmiany w modelach i synchronizować je z kodem źródłowym.

5. Ułatwienie dokumentacji projektu i jego późniejszego utrzymania

Dobrze zaprojektowane diagramy UML stanowią żywą dokumentację systemu. W przeciwieństwie do tradycyjnych dokumentów tekstowych, które szybko stają się nieaktualne, modele UML można łatwo aktualizować i utrzymywać w zgodności z rzeczywistym stanem systemu.

Najczęściej stosowane diagramy UML w praktyce

Diagram przypadków użycia (Use Case Diagram)

[Diagram przypadków użycia](#) to punkt wyjścia dla wielu projektów. Przedstawia funkcjonalności systemu z perspektywy użytkowników końcowych, definiując „co” system ma robić, bez wchodzenia w szczegóły „jak”. Jest idealny do komunikacji z interesariuszami biznesowymi i stanowi podstawę do dalszego modelowania.

Diagram klas (Class Diagram)

[Diagram klas](#) to serce modelowania obiektowego. Pokazuje strukturę systemu poprzez klasy, ich atrybuty, metody oraz relacje między nimi. Jest podstawowym narzędziem dla modelowania struktur danych.

Diagram aktywności (Activity Diagram)

[Diagram aktywności](#) wizualizuje przepływ sterowania w systemie, pokazując sekwencję akcji i decyzji. Jest szczególnie przydatny do modelowania złożonych procesów i algorytmów, oferując alternatywę dla tradycyjnych schematów blokowych.

Diagram sekwencji (Sequence Diagram)

[Diagram sekwencji](#) przedstawia interakcje między obiektami w czasie, pokazując kolejność wysyłanych komunikatów. Jest nieoceniony przy projektowaniu API, protokołów komunikacyjnych.

Diagram komponentów (Component Diagram)

[Diagram komponentów](#) pokazuje organizację systemu na poziomie komponentów programowych i ich zależności. Jest kluczowy przy projektowaniu architektury rozwiązania i mikroustug, pomagając w zarządzaniu zależnościami między modułami.

Diagram wdrożenia (Deployment Diagram)

[Diagram wdrożenia](#) ilustruje fizyczną architekturę systemu, pokazując rozmieszczenie komponentów na węzłach sprzętowych. Jest niezbędny przy planowaniu infrastruktury i strategii wdrażania aplikacji.

UML w kontekście współczesnych metodyk

Współczesne podejścia do modelowania biznesowego, takie jak opisane w artykułach [BPMN i UML w praktyce](#) oraz [BPMN i UML w praktyce cz.2 – zmiana](#), pokazują, że UML najlepiej sprawdza się w połączeniu z innymi notacjami. BPMN doskonale uzupełnia UML w obszarze modelowania procesów biznesowych, podczas gdy ArchiMate zapewnia kontekst architektoniczny na poziomie przedsiębiorstwa.

Praktyczne wskazówki dla rozpoczynających

Wybór właściwych diagramów UML zależy w dużej mierze od roli, jaką pełnimy w projekcie. Różne perspektywy wymagają różnych narzędzi modelowania.

Dla architektów systemów

Architekci powinni skoncentrować się przede wszystkim na **diagramach komponentów i sekwencji**. Diagram komponentów pozwala na zaprojektowanie modularnej struktury systemu, definiowanie granic komponentów oraz zarządzanie zależnościami między nimi. To kluczowe przy projektowaniu systemów skalowalnych i łatwych w utrzymaniu. Diagram sekwencji z kolei umożliwia architekcie precyzyjne zaprojektowanie komunikacji między komponentami, identyfikację potencjalnych wąskich gardeł wydajnościowych oraz optymalizację przepływu danych w systemie.

Dla analityków biznesowych

Analitycy znajdą największą wartość w **diagramach aktywności i klas**. Diagram aktywności doskonale nadaje się do modelowania złożonych procesów, pokazując przepływ decyzji, równoległe ścieżki wykonania oraz punkty synchronizacji. Pozwala to na precyzyjne zdefiniowanie logiki działania aplikacji i jej przekazanie zespołowi deweloperów. Diagram klas pomaga analitykom w strukturyzacji wiedzy domenowej, definiowaniu kluczowych encji biznesowych oraz ich wzajemnych relacji, co stanowi solidną podstawę do dalszego projektowania systemu.

Pamiętajmy, że celem modelowania nie jest tworzenie idealnych diagramów, ale efektywna komunikacja i lepsze zrozumienie systemu. UML to narzędzie, które służy ludziom, a nie odwrotnie – warto stosować je pragmatycznie, dostosowując poziom szczegółowości do potrzeb projektu i zespołu. Enterprise Architect oferuje doskonałe wsparcie dla wszystkich typów diagramów UML, umożliwiając również generowanie kodu i dokumentacji.

Modelowanie w UML to inwestycja w jakość i utrzymywalność oprogramowania, która zwraca się wielokrotnie w trakcie całego cyklu życia projektu.

UML w erze sztucznej inteligencji

W dobie rosnącej popularności projektów wykorzystujących AI i modele językowe (LLM), UML staje przed nowymi wyzwaniami i możliwościami. Tradycyjna notacja UML, zaprojektowana dla deterministycznych systemów programowych, nie jest w pełni

przystosowana do modelowania probabilistycznych zachowań AI czy złożonych pipeline'ów uczenia maszynowego.

Brakuje w UML dedykowanych konstrukcji do reprezentowania treningu modeli, walidacji danych, czy iteracyjnych procesów optymalizacji hiperparametrów. Diagramy sekwencji mogą nie oddawać specyfiki asynchronicznych procesów inferowania, gdzie czas odpowiedzi jest nieprzewidywalny, a wyniki mogą być probabilistyczne.

Niemniej jednak, UML nadal oferuje wartość w projektach AI. Diagramy przypadków użycia doskonale nadają się do definiowania scenariuszy interakcji z systemami AI, diagramy komponentów mogą modelować architektury MLOps, a diagramy aktywności – złożone przepływy przetwarzania danych. Kluczem jest ewolucja notacji i jej rozszerzenie o profile dedykowane dla AI, które mogłyby uwzględnić specyfikę systemów uczących się i adaptujących swoje zachowanie w czasie.

Przyszłość: Solution Designer jako uniwersalny twórca

Patrząc w przyszłość, możemy spodziewać się powstania nowej roli – solution designera, który będzie łączył kompetencje analityka biznesowego i architekta systemów.

Wspierany przez narzędzia AI, będzie uniwersalnym twórcą oprogramowania, zdolnym do przełożenia wymagań biznesowych na działające rozwiązania techniczne. UML, wraz z nadchodzącymi rozszerzeniami dla AI i automatyzacji, stanie się jego głównym językiem komunikacji. Diagramy UML będą służyć nie tylko do dokumentowania zamierzeń, ale także jako instrukcje dla systemów AI generujących kod, testy i dokumentację. Solution designer będzie orchestratorem całego procesu, wykorzystując modelowanie wizualne do kierowania pracą inteligentnych asystentów programistycznych.

Podsumowanie

Podsumowując, UML pozostaje fundamentalnym narzędziem w arsenale współczesnego inżyniera oprogramowania. Jego siła tkwi w standaryzacji komunikacji, jednoznaczności interpretacji oraz wsparciu całego cyklu życia oprogramowania. Różne diagramy UML odpowiadają na specyficzne potrzeby różnych ról w projekcie – od analityków skupiających się na procesach biznesowych, po architektów projektujących skalowalne systemy. Choć era AI stawia przed UML nowe wyzwania, jednocześnie otwiera możliwości jego ewolucji i zastosowania w jeszcze bardziej zaawansowanych scenariuszach. W przyszłości UML może stać się nie tylko językiem komunikacji między ludźmi, ale także instrukcjami dla inteligentnych systemów wspomagających tworzenie oprogramowania. Inwestycja w naukę modelowania UML to inwestycja w przyszłość – niezależnie od tego, jak bardzo zmieni się technologia, potrzeba jasnej komunikacji i precyzyjnego myślenia o systemach pozostanie aktualna.

Diagram przypadków użycia

Budowanie rozwiązań informatycznych poprzedza określenie potrzeb. Potrzeby te, zwane wymaganiami, pozwalają określić, jaki ma być system i jakie funkcje ma realizować. Wymagania w pierwszej fazie ich zbierania są zazwyczaj zapisywane słownie. Przy zastosowaniu języka UML mogą zostać również przedstawione w formie graficznej. Możliwość graficznego odwzorowania wymagań funkcjonalnych stawianych systemowi jest jedną z największych zalet języka UML. W tym wpisie przedstawimy technikę umożliwiającą modelowanie otoczenia systemu i stawianych mu wymagań.

Diagram przypadków użycia – definicja i zastosowanie

Diagram przypadków użycia (ang. use case diagram) to graficzna reprezentacja funkcjonalności systemu wraz z jego otoczeniem. Diagramy te pozwalają na prezentację właściwości i funkcji systemu z perspektywy użytkownika oraz zobrazowanie usług widocznych z zewnątrz systemu.

Diagram przypadków użycia – notacja i semantyka

Mimo że diagram przypadków użycia składa się z kilku elementów, odgrywa kluczową rolę w procesie projektowania systemu. Opisuje wymagania funkcjonalne, którym system musi sprostać, oraz otoczenie, w którym się znajduje. Diagram ten jest agregatem funkcji i usług wykonywanych przez system. Oprócz specyfikacji, pozwala na:

Identyfikację funkcjonalności

- Umożliwia jasne określenie granic systemu poprzez wizualizację interakcji między systemem a jego otoczeniem (aktorami).
- Pomaga w identyfikacji kluczowych funkcji systemu, które bezpośrednio odpowiadają na potrzeby użytkowników.
- Ułatwia priorytetyzację funkcjonalności poprzez analizę częstotliwości i znaczenia poszczególnych przypadków użycia.
- Wspomaga wykrywanie brakujących lub nadmiarowych funkcjonalności na wczesnym etapie projektowania.

Weryfikację postępów w modelowaniu i implementacji

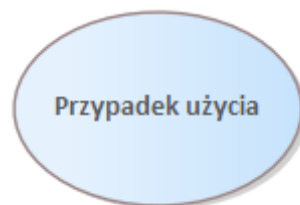
- Stanowi punkt odniesienia dla zespołu projektowego, umożliwiając śledzenie postępów w implementacji poszczególnych funkcjonalności.
- Pozwala na łatwe porównanie aktualnego stanu systemu z planowanymi funkcjonalnościami.

- Umożliwia iteracyjne rozwijanie systemu poprzez stopniowe dodawanie i uszczegóławianie przypadków użycia.
- Wspomaga proces testowania, dostarczając podstawy do tworzenia scenariuszy testowych opartych na przypadkach użycia.

Wspieranie komunikacji między uczestnikami projektu

- Dostarcza wspólnego języka dla różnych interesariuszy projektu (klientów, analityków, programistów, testerów).
- Ułatwia zrozumienie systemu przez osoby nietechniczne dzięki graficznej reprezentacji funkcjonalności.
- Stanowi podstawę do dyskusji i negocjacji zakresu projektu między klientem a zespołem deweloperskim.
- Pomaga w identyfikacji potencjalnych problemów i nieporozumień na wczesnym etapie projektu.
- Wspiera dokumentację projektu, dostarczając przejrzysty obraz funkcjonalności systemu.

Przypadek użycia



Rysunek 1. Przypadek użycia – notacja

Przypadek użycia (ang. use case) to zbiór scenariuszy powiązanych ze sobą wspólnym celem użytkownika. Jest graficzną reprezentacją wymagań funkcjonalnych. Definiuje zachowanie systemu bez informowania o jego wewnętrznej strukturze i narzucania sposobu implementacji.

Każdy przypadek użycia można opisać za pomocą następujących cech:

1. Nazwa
2. Opis
3. Przebieg zdarzeń (scenariusze)
4. Zależności i relacje
5. Diagramy aktywności
6. Wymagania specjalne
7. Warunki wstępne

8. Warunki końcowe

Scenariusze są jedną z najważniejszych cech przypadku użycia. Wskazują one sekwencję kolejno wykonywanych czynności służących do zrealizowania funkcjonalności zobrazowanej przez dany przypadek użycia.

Scenariusze należy traktować jako konkretne wystąpienia przypadku użycia.

Aktor



Rysunek 2. Aktor – notacja

Aktor (ang. actor) reprezentuje rolę, którą pełni użytkownik w stosunku do systemu oraz przypadków użycia. Aktor może być człowiekiem, urządzeniem lub innym systemem. Istotne jest, by pełnił określoną funkcję wobec systemu i przypadku użycia, którego używa.

Cechy aktora:

- Nie jest częścią systemu
- Reprezentuje rolę, w którą może wcielić się użytkownik
- Może reprezentować człowieka, urządzenie bądź inny system
- Może aktywnie wymieniać informacje z systemem
- Może dostarczać informacje

Szczególną uwagę należy zwrócić na fakt, iż aktor zawsze reprezentuje otoczenie systemu (nie jest częścią systemu) i musi być w interakcji chociaż z jednym przypadkiem użycia.

Połączenia w diagramie przypadków użycia

Asocjacja

Do łączenia przypadków użycia z aktorami najczęściej stosuje się powiązanie poprzez asocjację.



Rysunek 3. Połączenie aktora z przypadkiem użycia

Asocjacja wskazuje na to, że aktor korzysta z danej funkcji lub wchodzi w interakcję z systemem. Zazwyczaj odbiera jakieś dane lub dostarcza.

Przykładowy opis przypadku użycia: Rezerwacja samochodu

Aktor główny

Klient

Warunki wstępne

1. Klient jest zarejestrowany w systemie.
2. Klient jest zalogowany na swoje konto.

Warunki końcowe

1. Rezerwacja samochodu jest potwierdzona i zapisana w systemie.
2. Klient otrzymuje potwierdzenie rezerwacji.

Scenariusz główny

1. Klient wybiera opcję „Zarezerwuj samochód” w systemie.
2. System wyświetla formularz rezerwacji.
3. Klient wprowadza wymagane dane:
 - Data i godzina rozpoczęcia rezerwacji
 - Data i godzina zakończenia rezerwacji
 - Preferowana lokalizacja odbioru
 - Preferowana lokalizacja zwrotu
4. System sprawdza dostępność samochodów (include: Sprawdź Dostępność).
5. System wyświetla listę dostępnych samochodów wraz z cenami.
6. Klient wybiera konkretny samochód.
7. System wyświetla podsumowanie rezerwacji.
8. System weryfikuje dane klienta (include: Zweryfikuj Dane Klienta).
9. System oferuje opcję dodatkowego ubezpieczenia (extend: Wybierz Ubezpieczenie).
10. Klient potwierdza rezerwację.

11. System przekierowuje klienta do systemu płatności (include: Dokonaj Płatności).
12. Klient dokonuje płatności.
13. System potwierdza płatność.
14. System zapisuje rezerwację.
15. System generuje i wysyła potwierdzenie rezerwacji do klienta.

Scenariusze alternatywne

4a. Brak dostępnych samochodów:

1. System informuje klienta o braku dostępności.
2. System proponuje alternatywne daty lub lokalizacje.
3. Klient może zmodyfikować kryteria rezerwacji lub zakończyć proces.

8a. Weryfikacja danych klienta nie powiodła się:

1. System informuje klienta o problemie.
2. System prosi o aktualizację danych.
3. Klient aktualizuje dane lub kończy proces rezerwacji.

12a. Płatność nie powiodła się:

1. System informuje klienta o niepowodzeniu płatności.
2. System oferuje możliwość wyboru innej metody płatności.
3. Klient wybiera inną metodę płatności lub kończy proces rezerwacji.

Do opisania scenariuszy przypadków użycia o wiele lepiej sprawdzają się [diagramy aktywności](#).

Związki strukturalne: zawieranie i rozszerzenie

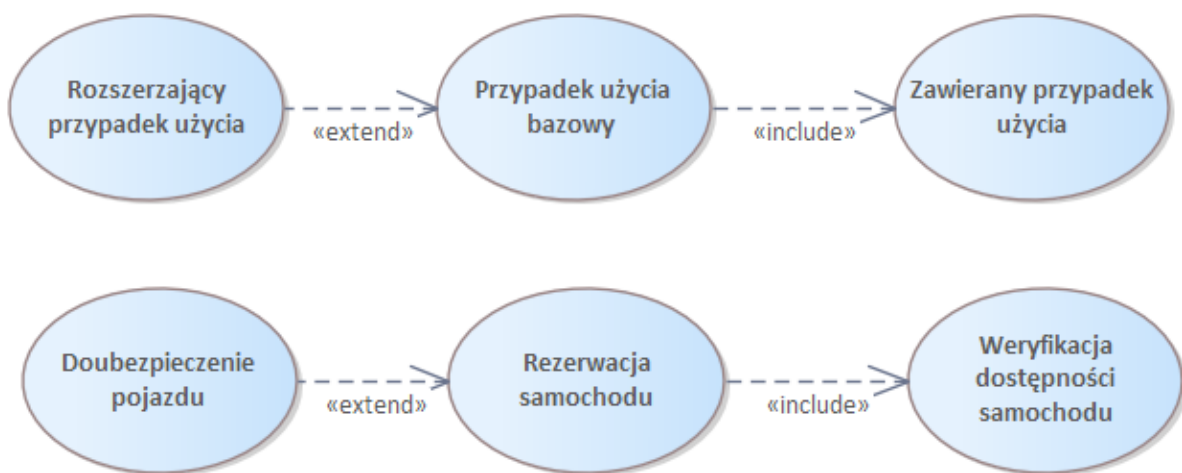
W diagramach przypadków użycia UML występują dwa główne typy związków strukturalnych: zawieranie (include) i rozszerzenie (extend). Oba te związki służą do modelowania relacji między przypadkami użycia, ale mają różne zastosowania i interpretacje.

Związek zawierania (ang. include):

- Reprezentuje obowiązkowe włączenie zachowania jednego przypadku użycia w inny.
- Stosowany, gdy część funkcjonalności jest współdzielona przez wiele przypadków użycia.
- Grot strzałki zawsze skierowany jest w stronę zawieranego przypadku użycia.
- Oznaczany stereotypem <<include>>.
- Czytany jako: „Przypadek użycia bazowy zawiera przypadek użycia zawarty”.

Związek rozszerzenia (ang. extend):

- Wskazuje, że dany przypadek użycia opcjonalnie rozszerza funkcjonalność bazowego przypadku użycia.
- Stosowany, gdy mamy do czynienia z wariantowym lub opcjonalnym zachowaniem.
- Funkcjonalność bazowego przypadku użycia jest rozszerzana po spełnieniu określonego warunku.
- Grot strzałki musi być skierowany w kierunku bazowego przypadku użycia.
- Oznaczany stereotypem <<extend>>.
- Czytany jako: „Przypadek użycia rozszerzający może rozszerzyć przypadek użycia bazowy”.



Rys. 4 Związki rozszerzenia i zawierania

Przedstawiony rysunek ilustruje oba typy związków na dwóch poziomach: ogólnym i konkretnym przykładzie.

Górna część rysunku (ogólna):

- „Przypadek użycia bazowy” jest centralnym elementem.
- Z lewej strony „Rozszerzający przypadek użycia” jest połączony z bazowym relacją <<extend>>.
- Z prawej strony „Zawierany przypadek użycia” jest połączony z bazowym relacją <<include>>.

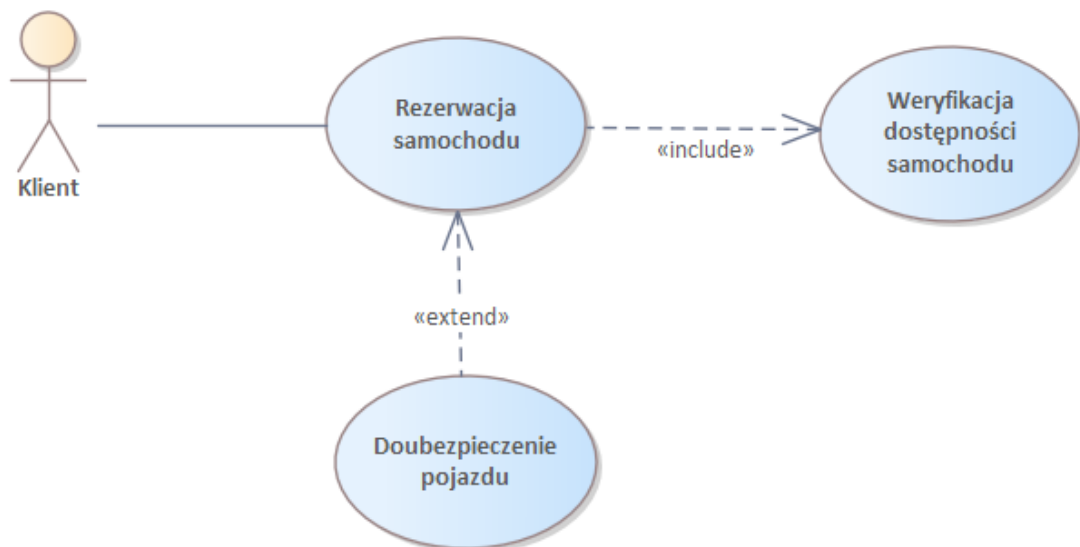
Dolna część rysunku (konkretny przykład):

- „Rezerwacja samochodu” jest centralnym (bazowym) przypadkiem użycia.
- „Doubezpieczenie pojazdu” jest połączone z „Rezerwacją samochodu” relacją <<extend>>, co oznacza, że jest to opcjonalna funkcjonalność, która może, ale nie musi być wykonana podczas rezerwacji.
- „Weryfikacja dostępności samochodu” jest połączona z „Rezerwacją samochodu” relacją <<include>>, co oznacza, że jest to obowiązkowa część procesu rezerwacji.

Jak czytać ten przykład:

1. Podstawowy proces to „Rezerwacja samochodu”.
2. Każda rezerwacja samochodu musi zawierać (include) weryfikację dostępności samochodu. Jest to nieodłączna część procesu rezerwacji.
3. Podczas rezerwacji samochodu klient może (ale nie musi) zdecydować się na doubezpieczenie pojazdu. Jest to opcjonalne rozszerzenie (extend) procesu rezerwacji.

Takie modelowanie pozwala na czytelne przedstawienie zarówno obowiązkowych, jak i opcjonalnych elementów procesu, co jest kluczowe dla zrozumienia pełnej funkcjonalności systemu rezerwacji samochodów.



Rys. 5 Diagram przypadków użycia – przykład

Należy także pamiętać, że każdy przypadek użycia musi być przyporządkowany minimum jednemu aktorowi, a każdy aktor przyporządkowany jest minimum jednemu przypadkowi użycia. Jeżeli przypadki użycia są połączone związkami zawierania lub rozszerzenia, to bazowy przypadek użycia staje się punktem łączącym aktora z danym przypadkiem użycia. Od tej reguły są wyjątki. Do wyjątków należą będą funkcje wewnętrzne aplikacji, które nie są widoczne z punktu widzenia otoczenia systemu.

Podsumowanie

Zrozumienie i prawidłowe stosowanie diagramów przypadków użycia, wraz z umiejętnością interpretacji związków include i extend, jest kluczowe dla efektywnego modelowania systemów i komunikacji w zespołach projektowych. Diagramy te stanowią pomost między wymaganiami biznesowymi a techniczną implementacją, zapewniając jasny obraz funkcjonalności systemu dla wszystkich zaangażowanych stron.

Diagram klas

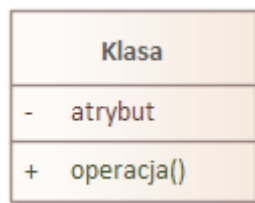
Celem wpisu jest prezentacja diagramu klas, który rysujemy, aby przedstawić statyczną strukturę systemu, ukazując klasy, ich atrybuty, metody oraz relacje między nimi, co stanowi podstawę do zrozumienia i implementacji oprogramowania.

Diagram klas – definicja i zastosowanie

Diagram klas obrazuje zbiór klas, interfejsów oraz związki między nimi. Diagram klas przedstawia statyczną strukturę systemu, podkreślając związki między klasami. W modelowaniu złożonych systemów można tworzyć wiele diagramów klas, skupiających się na różnych aspektach systemu. Kompleksowy model systemu powstaje przez złożenie wszystkich diagramów.

Diagram klas – notacja i semantyka

Klasa



Rys 1. Klasa

Klasa to opis zbioru obiektów o takich samych atrybutach i operacjach. Kluczowe aspekty klasy:

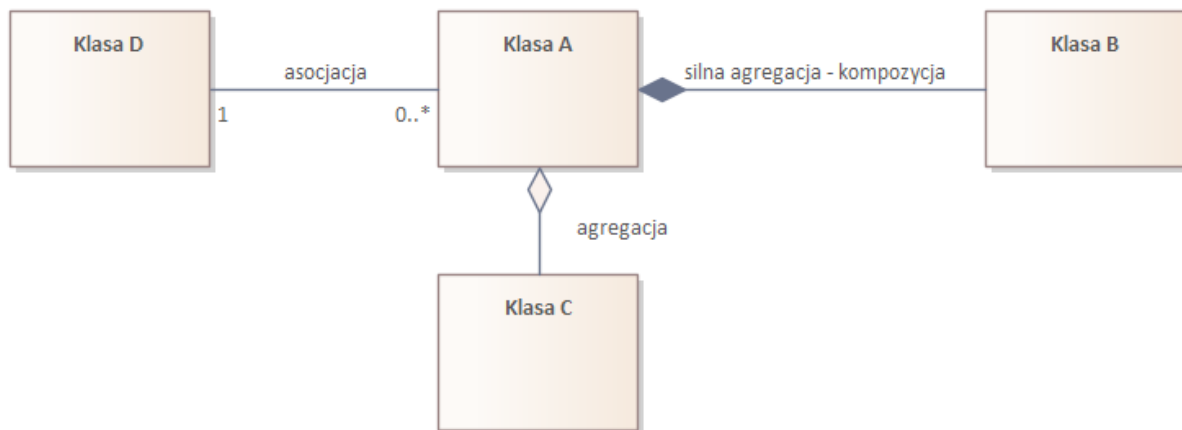
1. **Atrybuty:** Nazwane właściwości klasy, określające zestaw możliwych wartości dla instancji.
2. **Operacje:** Funkcje dostarczane przez obiekt, manifestujące się przez jego zachowanie.

Ponieważ klasa stanowi wzorzec dla tworzonych obiektów, nie jest celowe, aby każdy z nich przechowywał kopię tego samego opisu operacji; wszystkie utworzone obiekty powinny zatem odwoływać się do wspólnej definicji operacji zawartych w klasie. Warto zauważyć, że:

- Klasa stanowi wzorzec dla tworzonych obiektów.
- Obiekty odwołują się do wspólnej definicji operacji zawartych w klasie.
- Atrybuty charakteryzują pojedyncze obiekty lub ich grupy.
- Operacje działają na pojedynczych obiektach lub ich zbiorach.
- W niektórych przypadkach atrybuty lub operacje mogą być lepiej reprezentowane jako osobne klasy.

Diagramy klas stanowią fundamentalne narzędzie w modelowaniu obiektowym i UML, służąc do przedstawiania statycznej struktury systemu. Ich głównym zadaniem jest reprezentacja klas wraz z ich atrybutami i metodami oraz ilustracja różnorodnych relacji między klasami, takich jak asocjacje, agregacje, kompozycje i dziedziczenie. Dzięki temu diagramy klas umożliwiają skuteczną wizualizację struktury systemu, co znacząco ułatwia zrozumienie jego ogólnego działania.

Diagram klas powiązania



Rys 2. Podstawowe relacje stosowane na diagramie klas

Obecnie diagramy klas przeważnie są wykorzystywane do budowania diagramów mających na celu opisanie struktury przetwarzanych danych.

Powyższy diagram przedstawia fragment diagramu klas UML, pokazujący relacje między czterema klasami: A, B, C i D. Oto opis diagramu wraz z definicjami poszczególnych typów relacji:

1. Asocjacja (między Klasą D i Klasą A):
 - Oznaczenie: prosta linia
 - Krotność: 1 (Klasa D) do 0..* (Klasa A)
 - Definicja: Asocjacja to ogólny związek między klasami, oznaczający, że obiekty tych klas są w jakiś sposób powiązane ze sobą. W tym przypadku jeden obiekt Klasy D może być powiązany z wieloma obiektami Klasy A (lub z żadnym).
2. Silna agregacja – kompozycja (między Klasą A i Klasą B):
 - Oznaczenie: linia z wypełnionym rombem przy klasie zawierającej
 - Definicja: Kompozycja to specjalny rodzaj agregacji, gdzie część (Klasa B) nie może istnieć bez całości (Klasa A). Obiekty klasy składowej są tworzone i niszczone wraz z obiektem klasy zawierającej.
3. Agregacja (między Klasą A i Klasą C):

- Oznaczenie: linia z pustym rombem przy klasie zawierającej
- Definicja: Agregacja to relacja typu „całość-część”, gdzie część (Klasa C) może istnieć niezależnie od całości (Klasa A). Wskazuje, że obiekt jednej klasy zawiera obiekty innej klasy, ale nie są one ściśle zależne od siebie.

Dodatkowo, warto wspomnieć o innych typach relacji UML, które nie występują na tym diagramie:

4. Dziedziczenie (generalizacja):

- Oznaczenie: strzałka z pustym grotem wskazująca na klasę nadrzędną
- Definicja: Relacja między klasą ogólną (nadrzędną) a klasą bardziej szczegółową (podrzedną). Klasa podrzędna dziedziczy atrybuty i metody klasy nadrzędnej.

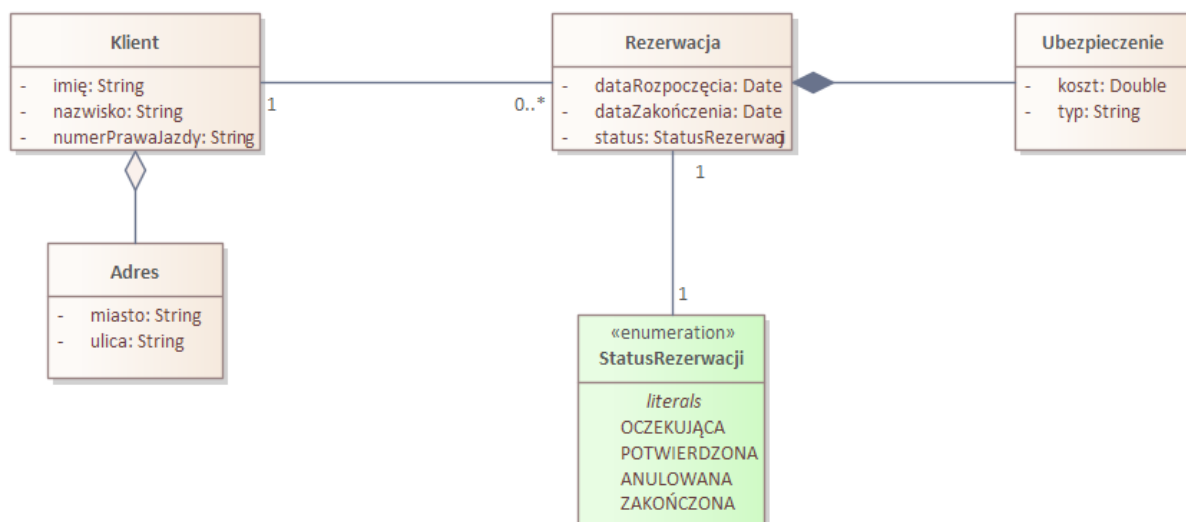
5. Realizacja (implementacja):

- Oznaczenie: przerywana linia ze strzałką wskazującą na interfejs
- Definicja: Wskazuje, że klasa implementuje określony interfejs, zobowiązując się do dostarczenia wszystkich metod zdefiniowanych w tym interfejsie.

6. Zależność:

- Oznaczenie: przerywana linia ze strzałką
- Definicja: Luźna relacja między klasami, gdzie zmiana w jednej klasie może wpłynąć na drugą, ale nie ma między nimi bezpośredniego powiązania strukturalnego.

Diagram klas przykład



Rys 3. Diagram klas – przykład

Powyższy diagram przedstawia fragment systemu rezerwacji dla wypożyczalni samochodów. Oto szczegółowy opis diagramu:

1. Klasa Klient:

- Atrybuty: imie (String), nazwisko (String), numerPrawaJazdy (String)
- Relacja: agregacja z klasą Adres (biały romb)
- Relacja: asocjacja z klasą Rezerwacja (linia prosta), krotność 1 do 0..* (jeden klient może mieć wiele rezerwacji lub żadnej)

2. Klasa Adres:

- Atrybuty: miasto (String), ulica (String)
- Jest agregowana przez klasę Klient

3. Klasa Rezerwacja:

- Atrybuty: dataRozpoczenia (Date), dataZakonczenia (Date), status (StatusRezerwacji)
- Relacja: kompozycja z klasą Ubezpieczenie (czarny romb), krotność 1 do 1 (każda rezerwacja ma dokładnie jedno ubezpieczenie)
- Relacja: asocjacja z enumeracją StatusRezerwacji, krotność 1 do 1

4. Klasa Ubezpieczenie:

- Atrybuty: koszt (Double), typ (String)
- Jest częścią kompozycji z klasą Rezerwacja

5. Enumeracja StatusRezerwacji:

- Wartości: OCZEKUJĄCA, POTWIERDZONA, ANULOWANA, ZAKOŃCZONA

Relacje na diagramie:

- Agregacja między Klient a Adres: sugeruje, że klient ma adres, ale adres może istnieć niezależnie od klienta.
- Asocjacja między Klient a Rezerwacja: pokazuje, że klient może dokonywać rezerwacji.
- Kompozycja między Rezerwacja a Ubezpieczenie: wskazuje, że ubezpieczenie jest integralną częścią rezerwacji i nie może istnieć bez niej.
- Asocjacja między Rezerwacja a StatusRezerwacji: określa, że każda rezerwacja ma przypisany status.

Diagram ten przedstawia kluczowe elementy systemu rezerwacji, pokazując jak klient, jego adres, rezerwacja i ubezpieczenie są ze sobą powiązane, oraz jak status rezerwacji jest reprezentowany w systemie.

Podsumowanie

Diagramy klas jako ważna część dokumentacji technicznej, przyczyniają się do lepszego zarządzania i utrzymania systemu w długiej perspektywie. W procesie projektowania oprogramowania diagramy klas odgrywają niezbędną rolę, pomagając w organizacji i zrozumieniu złożonych systemów poprzez przejrzyste przedstawienie ich struktury i wzajemnych powiązań między opisywanymi elementami.

Diagram aktywności

Modelowanie przepływu czynności jest jedną z najważniejszych technik oferowanych przez język UML. Wpis ten zawiera opis notacji i semantyki diagramu czynności, zwanego także diagramem aktywności.

Diagram czynności – definicja i zastosowanie

Diagram czynności (ang. activity diagram) jest diagramem behawioralnym, który służy do modelowania dynamicznych aspektów systemu. Jego zasadniczą funkcją jest przedstawienie sekwencji kroków, które są wykonywane przez modelowany fragment systemu. Jest idealny do opisania scenariuszy [przypadków użycia](#).

Diagramy aktywności w UML są kluczowym narzędziem do modelowania dynamicznych aspektów systemu, służącym do przedstawiania przepływu sterowania i danych poprzez sekwencje kroków i decyzji w procesach biznesowych lub operacjach systemowych.

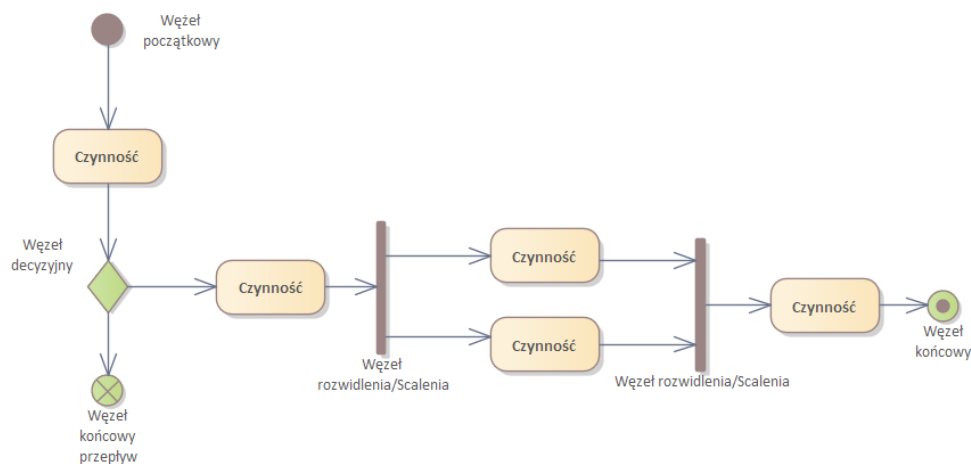
Diagramy te znajdują zastosowanie w

- przedstawianiu logiki algorytmów,
- wizualizacji przepływu pracy w systemie
- oraz dokumentacji przypadków użycia.

Ich główne zalety to przejrzyste przedstawienie złożonych procesów, łatwość zrozumienia dla interesariuszy nietechnicznych oraz możliwość modelowania równoległości i synchronizacji, choć mogą stać się skomplikowane przy bardzo złożonych procesach i nie pokazują szczegółów implementacyjnych.

Diagram czynności – notacja i semantyka

Na poniższym rysunku zgromadziłem podstawowe i najczęściej stosowane elementy notacji UML wykorzystywane na diagramach aktywności.



Rys 1. Podstawowe elementy stosowane na diagramach aktywności

Czynność (ang. action): Wykonywalny węzeł aktywności reprezentujący transformację lub proces modelowanego systemu. Może zmieniać stan systemu lub zwracać wynik. Czynność nie podlega dalszej dekompozycji.

Węzeł początkowy (ang. initial node): Element rozpoczynający aktywność. Każda aktywność może mieć tylko jeden węzeł początkowy.

Węzeł końcowy przepływu (ang. flow final node): Element sygnalizujący zatrzymanie sekwencji czynności przed jej całkowitym, zaplanowanym zrealizowaniem. Wskazuje sytuacje, gdy prezentowany scenariusz kończy się przedwcześnie.

Węzeł końcowy (ang. activity final node): Element kończący całą aktywność. Aktywność może mieć więcej niż jedno zakończenie.

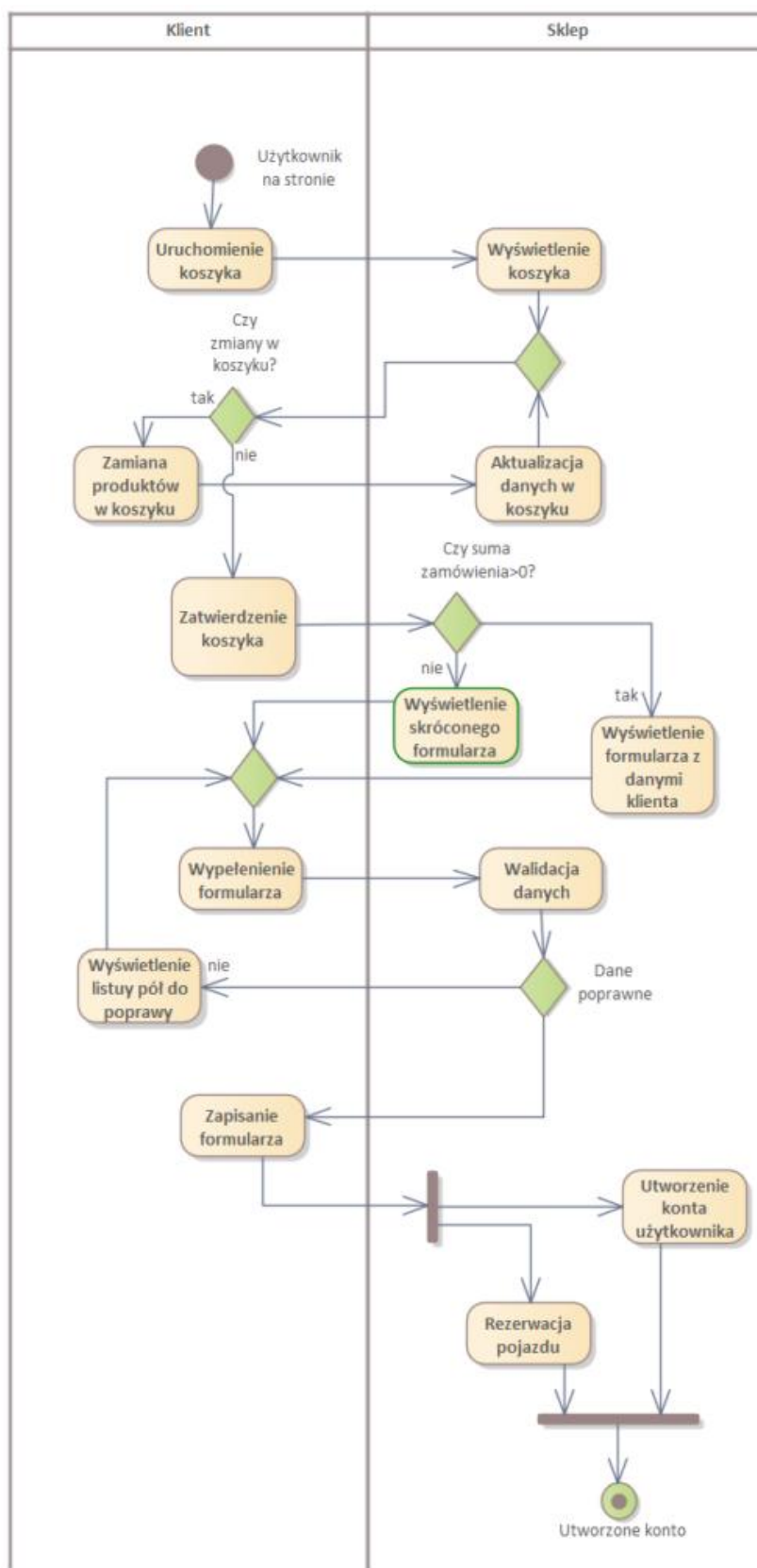
Węzeł decyzyjny (ang. decision node): Element umożliwiający wybór między kilkoma możliwościami, wprowadzający alternatywne ścieżki wykonania. Posiada jedno wejście i minimum dwa wyjścia z wykluczającymi się warunkami dozoru. Warunki powinny obejmować wszystkie możliwości, aby uniknąć zatrzymania przepływu. UML dopuszcza użycie warunku [else] dla domyślnej ścieżki.

Węzeł rozwidlenia (ang. fork node): Element inicjujący współbieżne wykonywanie czynności. Dzieli sekwencyjną ścieżkę na minimum dwie równoległe ścieżki wykonywane jednocześnie. Nie synchronizuje przepływów, a jedynie wskazuje początek równoległego wykonania.

Węzeł scalenia (ang. join node): Element łączący co najmniej dwa współbieżne przepływy, pełniący funkcję synchronizującą. Scala równoległe wykonywane przepływy w jeden. Liczba scalanych przepływów powinna odpowiadać liczbie węzłów wychodzących z powiązanego węzła rozwidlenia.

Elementy te, używane w odpowiedniej kombinacji, pozwalają na precyzyjne modelowanie złożonych procesów i przepływów w systemach, uwzględniając sekwencyjność, równoległość i warunkowe wykonanie czynności. Stosowane połączenie między tymi elementami to Control Flow.

Diagram czynności – przykład



Rys. 2 Diagram aktywności – przykład

Przedstawiony diagram przedstawia proces zakupowy w sklepie internetowym, który umożliwia wynajem pojazdu, podzielony na dwie partycje : „Klient” i „Sklep”. **Partycja aktywności (ang. activity partition)** jest to część diagramu czynności, która grupuje czynności realizowane przez konkretny byt (osoba albo aplikacja). Jest to diagram aktywności UML, który ilustruje przepływ czynności i decyzji w procesie zakupu.

Oto szczegółowy opis diagramu:

1. Proces rozpoczyna się od „Użytkownika na stronie”.
2. Klient uruchamia koszyk („Uruchomienie koszyka”), co prowadzi do wyświetlenia koszyka przez sklep.
3. Następnie pojawia się decyzja „Czy zmiany w koszyku?”:
 - Jeśli tak, klient dokonuje „Zmiany produktów w koszyku”, co skutkuje „Aktualizacją danych w koszyku” po stronie sklepu.
 - Jeśli nie, proces przechodzi do „Zatwierdzenia koszyka”.
4. Po zatwierdzeniu koszyka, sklep sprawdza „Czy suma zamówienia>0?”:
 - Jeśli tak, wyświetla „Wyświetlenie formularza z danymi klienta”.
 - Jeśli nie, wyświetla „Wyświetlenie skróconego formularza”.
5. Klient wypełnia formularz („Wypełnienie formularza”).
6. Sklep przeprowadza „Walidację danych”.
7. Jeśli dane są poprawne, proces przechodzi do „Zapisanie formularza”.
Jeśli nie, sklep wyświetla „Wyświetlenie listy pól do poprawy”, a klient musi ponownie wypełnić formularz.
8. Po zapisaniu formularza, proces rozdziela się na dwa równoległe działania:
 - „Utworzenie konta użytkownika” po stronie sklepu.
 - „Rezerwacja pojazdu” (co sugeruje, że może to być sklep z wypożyczalnią pojazdów).
9. Proces kończy się stanem „Utworzone konto”.

Diagram ten przedstawia kompleksowy proces zakupowy, uwzględniając różne scenariusze i interakcje między klientem a systemem sklepu internetowego, z naciskiem na obsługę koszyka, wprowadzanie danych klienta i finalizację zamówienia.

Podsumowanie

Dobre praktyki tworzenia diagramów aktywności obejmują zachowanie prostoty i czytelności, używanie opisowych nazw, grupowanie powiązanych czynności w podaktywności oraz unikanie nadmiernego skomplikowania. Diagramy te często uzupełniają inne diagramy UML, takie jak diagramy przypadków użycia, stanu i sekwencji, stanowiąc cenne narzędzie w analizie i projektowaniu systemów, pozwalające na efektywne komunikowanie dynamicznych aspektów systemu różnym interesariuszom projektu.

Diagram sekwencji

Niniejszy wpis prezentuje technikę modelowania przepływu sterowania z uwzględnieniem komunikatów w czasie. Celem jest omówienie diagramów sekwencji, które umożliwiają modelowanie interakcji pomiędzy obiektami wraz z komunikatami przepływającymi między nimi.

Diagram sekwencji – definicja i zastosowanie

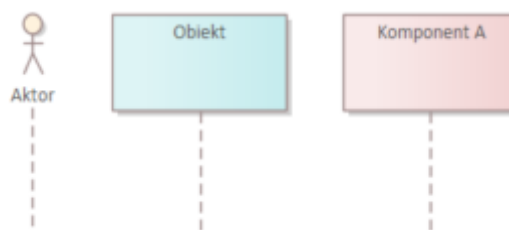
Diagram sekwencji (ang. sequence diagram) służy do prezentowania interakcji pomiędzy obiektami z uwzględnieniem czasowego aspektu komunikatów przesyłanych między nimi. Kluczowe cechy:

- Obiekty układane są wzdłuż osi X.
- Komunikaty przesyłane są wzdłuż osi Y, reprezentując upływ czasu.
- Główne zastosowanie: modelowanie zachowania systemu w kontekście scenariuszy przypadków użycia.
- Pozwalają odpowiedzieć na pytanie: jak przebiega komunikacja pomiędzy obiektami w czasie?
- Stanowią podstawową technikę modelowania zachowania systemu w realizacji przypadku użycia.

Diagram sekwencji – notacja i semantyka

W celu budowania diagramów sekwencji należy najpierw poznać elementy (klocki), z jakich te diagramy powstają. Poniżej zamieszczono notację wraz z opisem najważniejszych elementów służących do kreowania diagramów sekwencji.

Linia życia

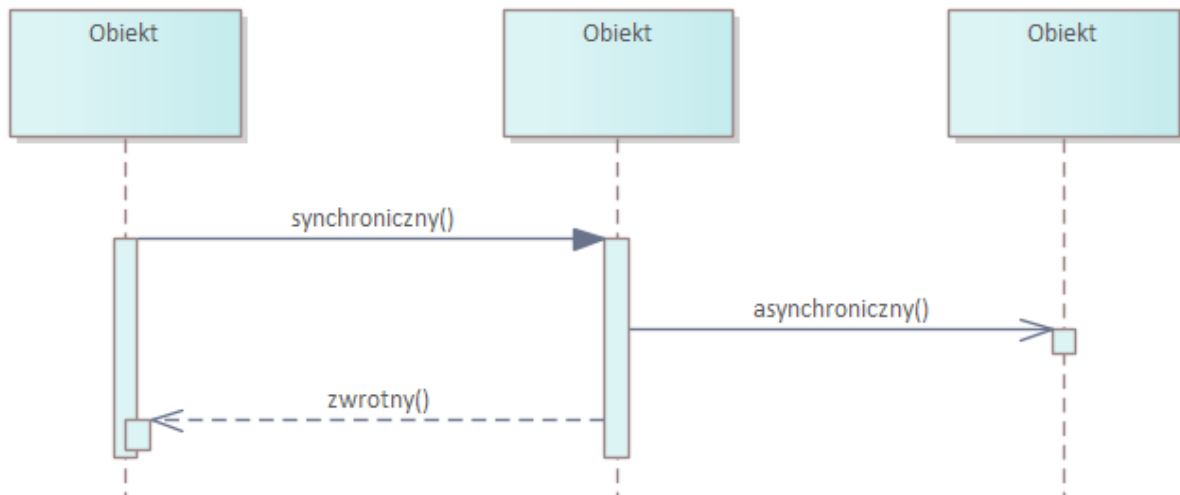


Rys 1. Linia życia – notacja

Linia życia to rola uczestnika interakcji, jaką pełni w czasie jej trwania. Linia życia reprezentuje współuczestnika interakcji i czas jego istnienia podczas realizacji scenariusza. Linie życia reprezentują konkretne byty – obiekty, systemy i mogą przyjmować stereotypy, które świadczą o roli, jaką pełni dany obiekt w systemie. Reprezentuje konkretne byty: obiekty, systemy (np.: komponenty)

Komunikat

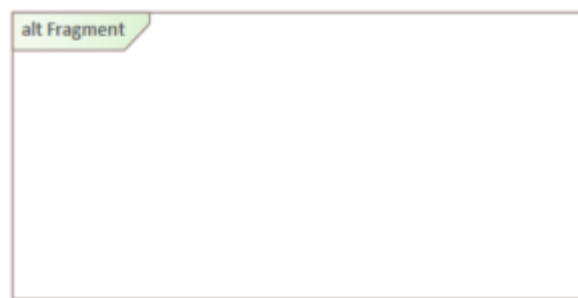
Komunikat (ang. message) jest to informacja przesyłana pomiędzy obiektami. W języku UML można korzystać z różnych typów komunikatów (rys. 2). Komunikat synchroniczny oznacza, że obiekt musi czekać na odpowiedź. Komunikat asynchroniczny nie wymaga oczekiwania na odpowiedź.



Rys. 2 Podstawowe rodzaje komunikatów na diagramach sekwencji

Stosując na diagramie komunikaty synchroniczne, przyjmuje się, że natychmiast po jego wywołaniu przychodzi odpowiedź. Taka konwencja pozwala na zmniejszenie liczby elementów na diagramie. W sytuacji, gdy modelowany fragment rzeczywistości jest inny niż założenia konwencji, należy umieścić komunikat zwrotny.

Fragment



Rys. 3 Fragment – notacja

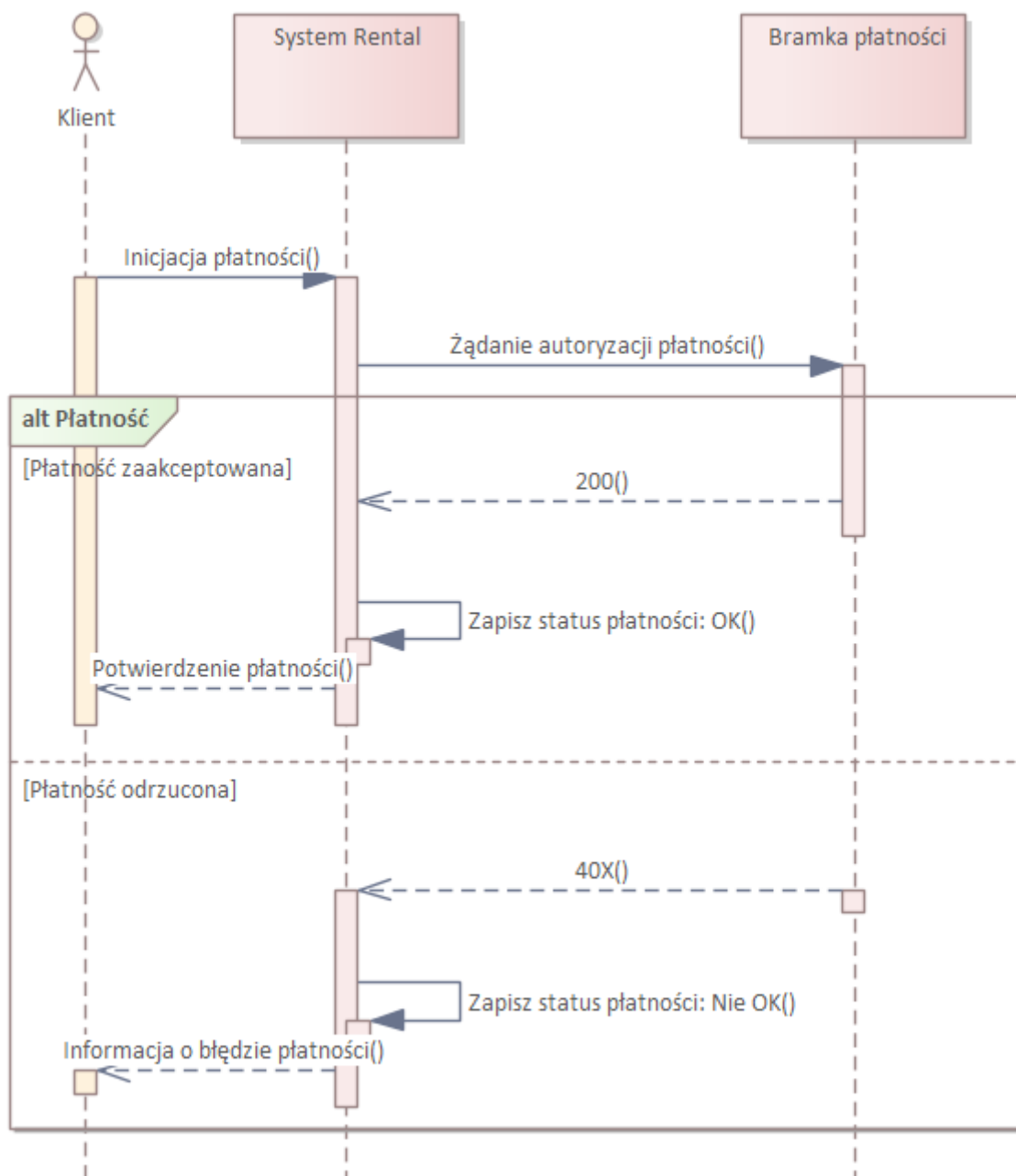
Fragment (ang. combined fragment) to conceptualnie zamknięta część diagramu sekwencji, która rozszerza możliwości obejmowanego przez siebie obszaru diagramu sekwencji. Fragment może zawierać w sobie pętle, powtórzenia, scenariusze alternatywne lub wskazywać poziom abstrakcji modelowanego fragmentu systemu.

Rodzaj fragmentu jest określany poprzez umieszczenie odpowiedniego słowa kluczowego w lewym górnym rogu. Poniżej opis wszystkich słów kluczowych, które mogą wystąpić we fragmentach. Najpopularniejsze typy to:

- alt – dzieli fragment interakcji zgodnie z warunkami logiki Boole’a na dwa alternatywne scenariusze; każda z alternatyw musi być opatrzona warunkiem dozoru, którego spełnienie gwarantuje wykonanie danej alternatywy.
- loop – powtórzenie fragmentu interakcji określoną warunkiem liczbę razy.
- par – prezentuje równoległe wykonywanie przepływu wiadomości.

Możliwość stosowania fragmentów w diagramach sekwencji pomaga przy modelowaniu scenariuszy.

Diagram sekwencji przykład



Rys. 4 Diagram sekwencji – przykład

Główne elementy diagramu to:

1. Klient – inicjator procesu płatności
2. System Rental – system obsługujący wynajem
3. Bramka płatności – zewnętrzny system autoryzacji płatności

Przebieg interakcji:

1. Klient inicjuje płatność.
2. System Rental wysyła żądanie autoryzacji płatności do bramki płatności.
3. Bramka płatności przetwarza żądanie i zwraca odpowiedź. Diagram pokazuje dwa możliwe scenariusze:
 - a) Płatność zaakceptowana:
 - Bramka zwraca kod 200 (sukces)
 - System Rental zapisuje status płatności jako OK
 - System potwierdza płatność klientowi
 - b) Płatność odrzucona:
 - Bramka zwraca kod 40X (błąd)
 - System Rental zapisuje status płatności jako Nie OK
 - System informuje klienta o błędzie płatności

Diagram pokazuje asynchroniczną komunikację między systemami, co jest widoczne przez przerywane linie strzałek powrotnych. Prezentuje on również alternatywne ścieżki wykonania (zaakceptowana vs odrzucona płatność) za pomocą fragmentu oznaczonego jako „alt”.

Warto zauważyć, że diagram ten przedstawia uproszczony proces płatności, skupiając się na kluczowych interakcjach między uczestnikami systemu.

Podsumowanie

Diagramy sekwencji są potężnym narzędziem do modelowania dynamicznych aspektów systemu. Pozwalają na szczegółowe przedstawienie interakcji między obiektami w czasie, co jest kluczowe dla zrozumienia i projektowania złożonych systemów informatycznych. Ich elastyczność w reprezentowaniu różnych scenariuszy, w tym alternatywnych ścieżek i powtórzeń, czyni je nieocenionym narzędziem w procesie analizy i projektowania oprogramowania.

Diagram komponentów

Celem tego wpisu jest przedstawienie technik projektowych związanych z prezentacją komponentów w systemach informatycznych. Skupię się na diagramie komponentów, jego definicji, zastosowaniu oraz kluczowych elementach notacji.

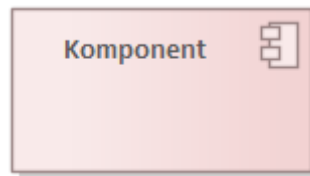
Diagram komponentów – definicja i zastosowanie

Diagram komponentów (ang. component diagram) to narzędzie służące do ilustrowania organizacji i zależności pomiędzy komponentami systemu. Charakteryzuje się następującymi cechami:

1. Przedstawia system na wyższym poziomie abstrakcji niż diagram klas.
2. Każdy komponent może być implementacją jednej lub większej liczby klas.
3. Służy do określania szczegółów niezbędnych do budowy systemu.
4. Pomaga w zrozumieniu struktury systemu i relacji między jego częściami.

Diagram komponentów – notacja i semantyka

Komponent



Rys.1 Komponent

Komponent (ang. component) to modułarny – logiczny bądź fizyczny – fragment systemu. Charakteryzuje się następującymi cechami:

- Zapewnia bardziej zwięzły opis zachowania systemu niż jego szczegółowa implementacja.
- Może być stosowany w dwóch aspektach: a) Definiowanie zewnętrznego oblicza systemu. b) Implementacja funkcjonalności systemu.
- Służy jako agregat dla podsystemów na wszystkich poziomach systemu.
- Użyty ze stereotypem <<subsystem>>, oznacza agregat dla komponentów dużej skali.

Komponent zazwyczaj reprezentuje aplikację, moduł aplikacji, albo każdy inny element grup logiczny bądź fizyczny – fragment systemu.

Interfejs

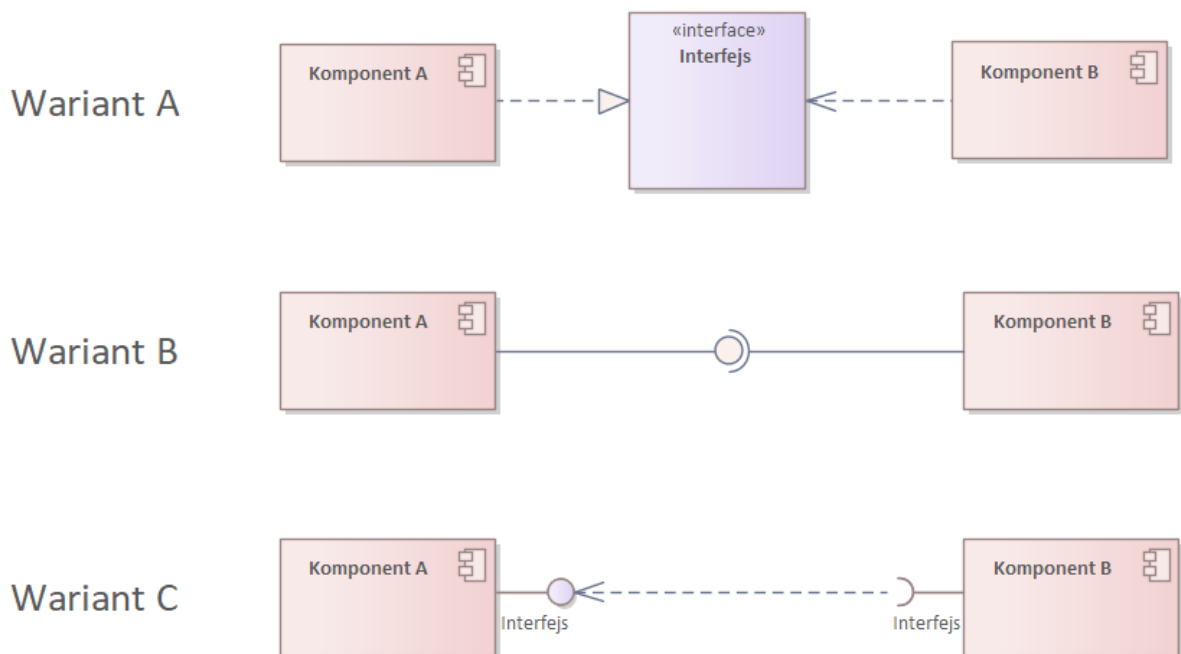


Rys. 2 Interfejs

Interfejs (ang. interface) to zestaw operacji definiujących usługi oferowane przez komponent (lub klasę). Kluczowe cechy interfejsu:

- Służy do prezentowania komunikacji pomiędzy komponentami.
- Określa kontrakt, który komponent musi spełnić.
- Umożliwia luźne powiązanie między komponentami, promując modularność i elastyczność systemu.

Powiązania i relacje na diagramach komponentów



Rys. 3 Diagram komponentów – powiązania

Powyższy rysunek przedstawia trzy różne warianty (A, B i C) ilustrujące sposoby reprezentacji powiązań między komponentami w diagramie komponentów UML. Oto szczegółowy opis każdego wariantu:

Wariant A:

- Pokazuje dwa komponenty: Komponent A i Komponent B.
- Między nimi znajduje się element oznaczony jako <<interface>> Interfejs.
- Komponent A dostarcza (realizuje/wystawia) interfejs.
- Komponent B połączony jest z interfejsem za pomocą relacji zależności (ang. dependency), która świadczy o tym, że korzysta z tego interfejsu.
- Ten wariant jasno pokazuje, że interfejs jest oddzielnym bytem, z którego korzystają oba komponenty.

Wariant B:

- Przedstawia bezpośrednie połączenie między Komponentem A i Komponentem B.
- Połączenie to jest reprezentowane przez ciągłą linię z okrągłym symbolem (przypominającym „lizak”) po stronie Komponentu A. Jest to relacja Assembly.
- Ten symbol oznacza, że Komponent A udostępnia interfejs, z którego korzysta Komponent B.
- Jest to bardziej zwięzła reprezentacja relacji interfejsu, bez jawnego pokazywania elementu interfejsu.

Wariant C:

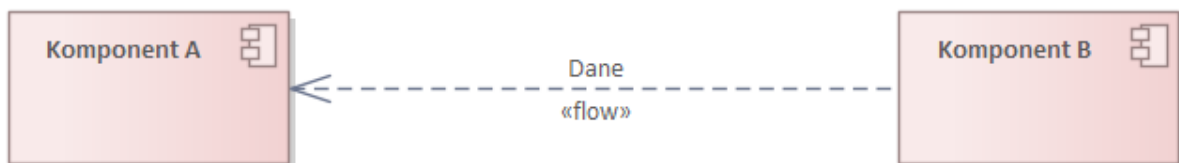
- Podobnie jak w wariacie B, pokazuje bezpośrednie połączenie między Komponentem A i Komponentem B.
- Tym razem używa symbolu „gniazda” (półokrąg) po stronie Komponentu B i symbolu „lizak” po stronie Komponentu A.
- Symbol „lollipop” oznacza dostarczany interfejs (Komponent A dostarcza interfejs).
- Symbol „gniazda” oznacza wymagany interfejs (Komponent B wymaga interfejsu).
- Połączenie jest reprezentowane przez przerywaną linię – zależność (ang. dependency).
- Ta notacja jasno pokazuje, który komponent wymaga interfejsu, a który go dostarcza.

Powiązania:

1. We wszystkich wariantach Komponent A jest powiązany z Komponentem B poprzez interfejs.
2. Wariant A pokazuje to powiązanie explicite poprzez osobny element interfejsu.
3. Warianty B i C przedstawiają to powiązanie w bardziej zwięzły sposób, bez osobnego elementu interfejsu.

4. We wszystkich przypadkach idea jest taka sama: Komponent B korzysta z interfejsu dostarczanego przez Komponent A.

Te trzy warianty ilustrują różne sposoby reprezentacji tej samej koncepcji w diagramach komponentów UML, pozwalając na wybór najbardziej odpowiedniej notacji w zależności od potrzeb diagramu i preferencji projektanta. Przez wiele lat stosowałem wariant C. Dziś oręduję za uproszczeniem notacji Wariant B to mój ulubiony wariant, gdzie w nazwie relacji Assembly wpisuję nazwę interfejsu a w stereotypie jego typ. Należy także pamiętać, że jest stosowana jest odmiana diagramu komponentów, gdzie jest użyta relacja flow. Jego celem jest pokazanie przepływu danych.

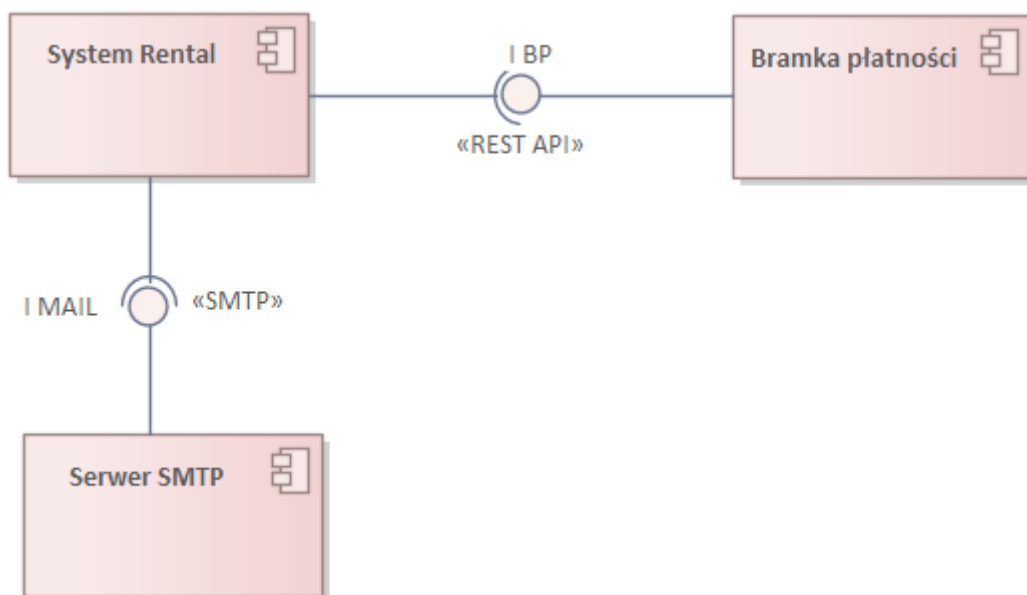


Rys. 4 Przepływ danych na diagramach komponentów

Diagram komponentów – przykład

Poniższy rysunek przedstawia diagram komponentów dla systemu wynajmu (Rental) i jego interakcji z zewnętrznymi usługami. Oto szczegółowy opis:

1. System Rental – wspiera wynajem pojazdów
2. Bramka płatności – umożliwi płatności kartą kredytową za wynajem
3. Serwer SMTP:
 - Trzeci komponent na diagramie, oznaczony jako „Serwer SMTP”.
 - Również posiada symbol komponentu.



Rys. 5 Diagram komponentów – przykład

Połączenia i interfejsy na rysunku to:

a) Między System Rental a Bramką płatności:

- Połączenie oznaczone jako „IBP” („Interfejs Bramki Płatności”).
- Używa notacji „litzak” (kółko na końcu linii) po stronie Bramki płatności, co oznacza, że Bramka płatności dostarcza ten interfejs.
- Połączenie opisane jako «REST API», co wskazuje na typ komunikacji między tymi komponentami.

b) Między System Rental a Serwerem SMTP:

- Połączenie oznaczone jako „IMAIL”.
- Używa notacji „litzak” po stronie Serwera SMTP, co oznacza, że Serwer SMTP dostarcza ten interfejs.
- Połączenie opisane jako «SMTP», co wskazuje na protokół używany do komunikacji.

Diagram ten ilustruje architekturę komponentową systemu wynajmu, pokazując jak główny system integruje się z zewnętrznymi usługami płatności i wysyłki e-maili.

Wykorzystanie interfejsów (IBP i IMAIL) oraz określenie protokołów komunikacji (REST API i SMTP) jasno definiuje sposób interakcji między komponentami. Taka reprezentacja jest typowa dla diagramów komponentów w UML, pokazując strukturę systemu na wyższym poziomie abstrakcji i koncentrując się na głównych elementach oraz ich wzajemnych zależnościach.

Podsumowanie

Diagram komponentów jest cennym narzędziem w procesie projektowania i rozwoju systemów informatycznych. Pozwala na wizualizację struktury systemu na poziomie komponentów, co jest szczególnie użyteczne w przypadku dużych i złożonych projektów.

Diagram wdrożenia

Celem niniejszego wpisu jest zaprezentowanie techniki, która wspomaga modelowanie fizycznych aspektów rozwiązań informatycznych, ze szczególnym uwzględnieniem diagramu wdrożenia w języku UML.

Diagram wdrożenia – definicja i zastosowanie

Diagram wdrożenia (ang. Deployment Diagram) służy do obrazowania konfiguracji poszczególnych węzłów fizycznych działającego systemu i zainstalowanych na nich komponentów.

Na diagramie wdrożenia prezentuje się węzły fizyczne systemu, którymi są komputery, serwery i inny sprzęt komputerowy funkcjonujący w działającym systemie, oraz związki między nimi. Sprzęt fizyczny przedstawiany na diagramie wdrożenia może obejmować:

1. Serwery:

- Serwery aplikacji (np. Apache Tomcat, JBoss)
- Serwery baz danych (np. MySQL, Oracle, Microsoft SQL Server)
- Serwery WWW (np. Apache, Nginx)
- Serwery plików
- Serwery poczty elektronicznej

2. Komputery klienckie:

- Komputery stacjonarne
- Laptopy
- Terminale

3. Urządzenia mobilne:

- Smartfony
- Tablety
- Urządzenia IoT (Internet of Things)

4. Urządzenia sieciowe:

- Routery
- Przetączniki (switche)
- Firewalle
- Load balancery

5. Urządzenia peryferyjne:

- Drukarki
- Skanery
- Urządzenia wielofunkcyjne

6. Systemy pamięci masowej:

- Macierze dyskowe
- Systemy NAS (Network Attached Storage)
- Systemy SAN (Storage Area Network)

7. Urządzenia specjalistyczne:

- Systemy kontroli dostępu
- Kasy fiskalne
- Terminale płatnicze
- Systemy monitoringu

8. Urządzenia wbudowane:

- Sterowniki PLC
- Systemy SCADA
- Urządzenia przemysłowe

Związki między tymi elementami mogą reprezentować:

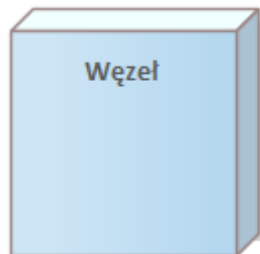
- Połączenia sieciowe (np. LAN, WAN, VPN)
- Protokoły komunikacyjne (np. HTTP, TCP/IP, FTP)
- Zależności funkcjonalne (np. klient-serwer, master-slave)

Diagram wdrożenia pozwala na zobrazowanie, jak różne komponenty oprogramowania są rozlokowane na fizycznych urządzeniach, co jest kluczowe dla zrozumienia architektury systemu, planowania wydajności, bezpieczeństwa oraz strategii utrzymania i rozwoju infrastruktury IT.

Diagram wdrożenia – notacja i semantyka

W niniejszym wpisie skupię się tylko na najważniejszych elementach wchodzących w skład diagramów wdrożenia.

Węzeł



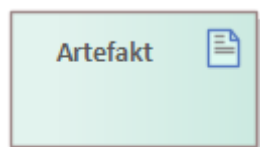
Rys. 1 Węzeł

Węzeł (ang. Node) to działający fizycznie składnik systemu. Węzeł zazwyczaj ma pewne zdolności przetwarzania i pamięć. Reprezentuje zasoby obliczeniowe. Na diagramach wdrożenia węzły stanowią fizyczne byty, na których rezyduje budowany system.

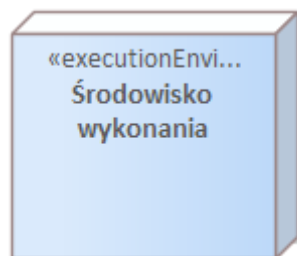
Artefakt

Artefakt (ang. Artifact) to fizycznie istniejący zasób informatyczny w postaci takich bytów jak: model, plik lub tabela. Jest używany lub wytwarzany w czasie wdrażania oprogramowania. Artefakt może posiadać atrybuty i operacje i może być połączony asocjacją z innym artefaktem.

Środowisko wykonania



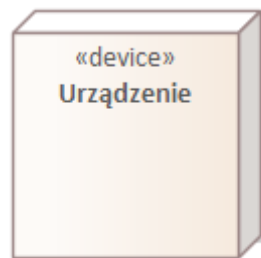
Rys. 2 Artefakt



Rys. 3 Środowisko wykonania

Środowisko wykonania (ang. Execution Environment) to element prezentujący wybrany rodzaj platformy, np. system operacyjny, silnik bazy danych i inne. Jest zazwyczaj częścią innego węzła, który modeluje sprzęt.

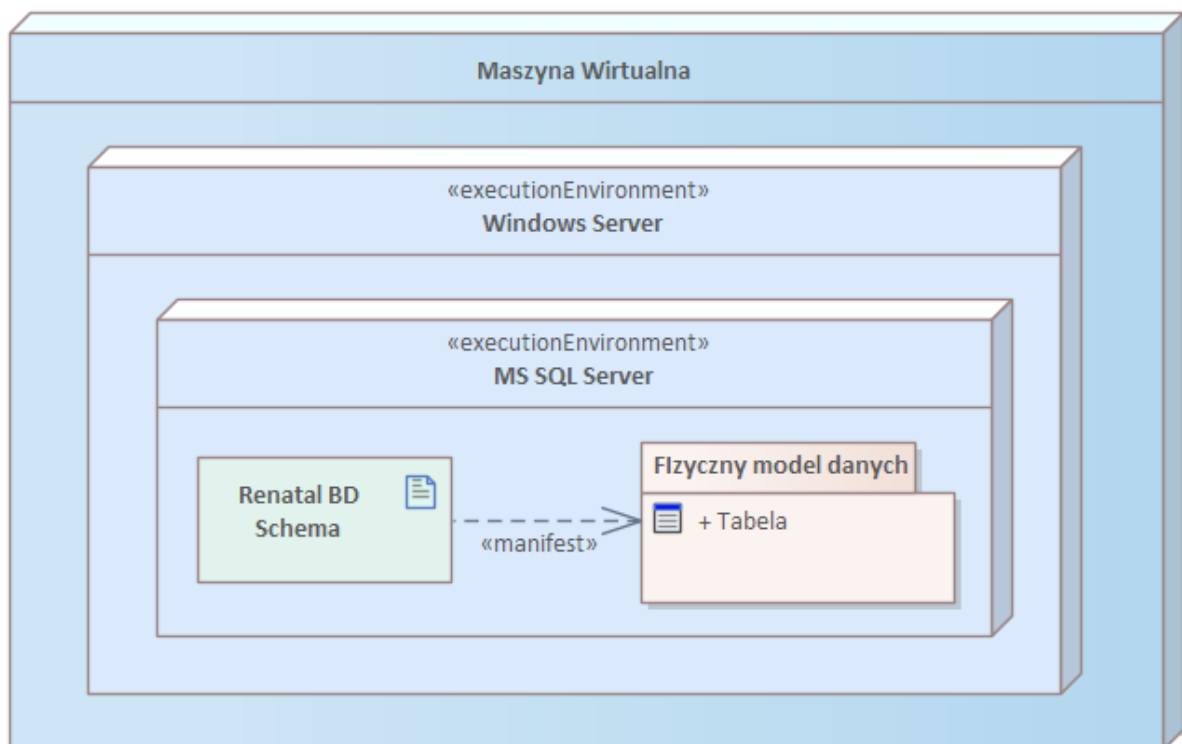
Urządzenie



Rys. 4 Urządzenie

Urządzenie (ang. Device) stanowi fizyczne zasoby komputerowe, na których mogą być wykonane wdrażane artefakty. Różnica pomiędzy zwykłym węzłem a urządzeniem jest nieznaczna, z naciskiem na używanie urządzenia w sytuacjach, gdy przedstawiany element jest typową maszyną – sprzętem komputerowym. Często w urządzeniu zagnieżdżone jest środowisko wykonania do specyfikowania oprogramowania.

Diagram wdrożenia – przykładowe zastosowanie



Rysunek 5. Diagram wdrożenia – struktura węzła

Powyższy rysunek przedstawia diagram wdrożenia systemu, który wykorzystuje maszynę wirtualną i środowiska wykonawcze. Oto szczegółowy opis:

1. Maszyna Wirtualna:

- Jest to najwyższy poziom diagramu, reprezentujący wirtualne środowisko, na którym zainstalowane są pozostałe komponenty.

2. Windows Server:

- Jest to pierwsza warstwa środowiska wykonawczego (executionEnvironment) wewnątrz maszyny wirtualnej.
- Reprezentuje system operacyjny, na którym uruchamiane są pozostałe komponenty.

3. MS SQL Server:

- To druga warstwa środowiska wykonawczego, zagnieżdżona wewnątrz Windows Server.
- Reprezentuje system zarządzania bazą danych Microsoft SQL Server.

4. Renatal BD Schema:

- Jest to artefakt reprezentujący schemat bazy danych, prawdopodobnie dla systemu wynajmu (rental).
- Znajduje się wewnątrz środowiska MS SQL Server.

5. Fizyczny model danych:

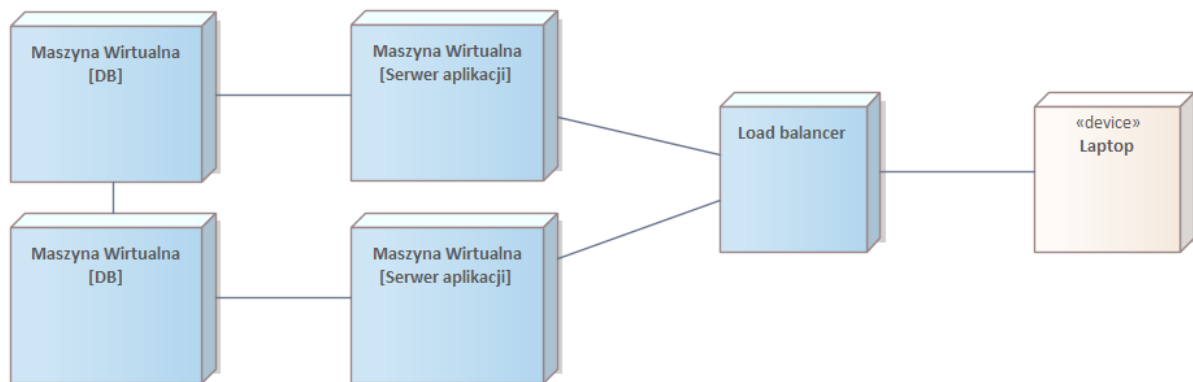
- To kolejny artefakt, również znajdujący się wewnątrz środowiska MS SQL Server.
- Zawiera element oznaczony jako „+ Tabela”, co sugeruje, że reprezentuje fizyczną strukturę danych w bazie.

6. Relacja „manifest”:

- Istnieje strzałka z opisem „«manifest»” łącząca „Renatal BD Schema” z „Fizyczny model danych”.
- Ta relacja wskazuje, że schemat bazy danych jest manifestowany (realizowany) w fizycznym modelu danych.

Diagram ten pokazuje warstwową strukturę środowiska, gdzie maszyna wirtualna zawiera system operacyjny Windows Server, który z kolei hostuje MS SQL Server. Wewnątrz serwera SQL znajdują się artefakty reprezentujące schemat bazy danych i jej fizyczną implementację. Całość ilustruje typowe wdrożenie systemu bazodanowego w środowisku wirtualnym, z wyraźnym podziałem na warstwy logiczne i fizyczne.

Powyższy rysunek przedstawia diagram wdrożenia dla systemu rozproszonego, ilustrujący architekturę wysokiej dostępności. Oto szczegółowy opis:



Rys. 6 Diagram wdrożenia – połączenia między węzłami

1. Maszyny Wirtualne [DB]:

- Widoczne są dwie maszyny wirtualne oznaczone jako [DB], co sugeruje, że są to serwery baz danych.
- Umieszczenie dwóch maszyn DB wskazuje na redundancję, prawdopodobnie w celu zapewnienia wysokiej dostępności lub rozłożenia obciążenia.

2. Maszyny Wirtualne [Serwer aplikacji]:

- Widoczne są również dwie maszyny wirtualne oznaczone jako [Serwer aplikacji].
- Podobnie jak w przypadku baz danych, duplikacja serwerów aplikacji sugeruje redundancję i możliwość rozłożenia obciążenia.

3. Load balancer:

- Pomiędzy serwerami aplikacji a klientem znajduje się element oznaczony jako „Load balancer”.
- Jego rolą jest równoważenie obciążenia między serwerami aplikacji, co poprawia wydajność i niezawodność systemu.

4. Laptop:

- Po prawej stronie diagramu widoczny jest element oznaczony jako „«device» Laptop”.
- Urządzenie klienckie, z którego użytkownicy łączą się z systemem.

5. Połączenia:

- Widoczne są linie łączące poszczególne elementy, reprezentujące połączenia sieciowe między komponentami systemu.
- Maszyny DB są połączone z odpowiadającymi im serwerami aplikacji.
- Serwery aplikacji są połączone z load balancerem.
- Load balancer jest połączony z laptopem klienta.

Ten diagram ilustruje typową architekturę systemu o wysokiej dostępności, z redundancją na poziomie baz danych i serwerów aplikacji, oraz z wykorzystaniem load balancera do optymalizacji ruchu. Taka struktura zapewnia odporność na awarie, możliwość skalowania oraz efektywne zarządzanie obciążeniem, co jest kluczowe dla systemów wymagających ciągłości działania i obsługi dużej liczby użytkowników.

Podsumowanie

Modelowanie infrastruktury, w tym wykorzystanie diagramów wdrożenia, nie jest tylko technicznym aspektem rozwoju oprogramowania. Jest to strategiczne narzędzie, które wspiera cały cykl życia systemu informatycznego, od planowania przez implementację, aż po utrzymanie i rozwój. Pozwala na podejmowanie świadomych decyzji, optymalizację zasobów i efektywne zarządzanie ryzykiem w złożonych środowiskach IT.